THE SENTINEL HYBRID-SOC

R3: Threat Intelligence | R4: Risk Management | R5: Security Monitoring & Incident Response

Student: Yessy Vasquez

Program: DAE Cybersecurity Cohort — Stafford, CT

Platform: Apple Mac M4 | UTM | Kali Linux ARM64 | Docker

Date: April 2026

RUBRIC R3 — Implement Threat Intelligence Principles: Analysis of 2 IoCs with detection methods, OpenCTI platform deployment via Docker with 2 connectors configured, platform setup documentation, and evidence of functionality.

RUBRIC R4 — Develop and Apply Risk Management Strategies: 2 critical risks from vulnerability scan results with explanations and mitigation steps, plus 1 risk monitoring procedure with justification.

RUBRIC R5 — Implement Security Monitoring and Incident Response: 1 monitoring use case with detection rules and alert prioritization, 1 incident response scenario with classification, response steps, and lessons learned.

RUBRIC R3 — Implement Threat Intelligence Principles

Rubric R3 — Complete

This section documents threat intelligence implementation through two components: analysis of two real Indicators of Compromise (IoCs) extracted from the clientarm4n.elf malware analysis, and deployment of the OpenCTI Threat Intelligence Platform via Docker with two active connectors.

Part 1 — Indicators of Compromise (IoC) Analysis

An Indicator of Compromise is a specific, measurable forensic artifact that signals with high confidence that a system has been compromised. IoCs are directly actionable — they can be imported into SIEM rules, EDR platforms, and firewall blocklists.

IoC 1 — MD5 File Hash (Static IoC)

IoC Attribute	Detail
Type	File Hash (MD5)
Value	4806a550a504c1991566ebe2987b800e
Source	Joe Sandbox Static Analysis — Analysis ID: 1894052
Detection Method	Static file analysis and hash comparison against VirusTotal, MalwareBazaar, and internal threat databases. The file was identified as an ELF ARM binary with malicious classification.
Confidence Level	HIGH — directly confirmed malicious by Joe Sandbox classification (32/100 MALICIOUS)
Threat Indication	This MD5 hash is a unique fingerprint for the clientarm4n.elf binary. Its presence on any system — regardless of filename — confirms infection by this specific botnet agent. The hash was cross-referenced against global threat databases and confirmed malicious.
SIEM Detection Rule	ALERT IF: file.hash.md5 = '4806a550a504c1991566ebe2987b800e' → severity=CRITICAL, action=isolate_host
Limitation	Hash-based IoCs have limited lifespan. Recompiling the binary changes the hash. Must be paired with behavioral IoCs for long-term protection.

IoC 2 — Malicious Network Behavior (Behavioral IoC)

IoC Attribute	Detail
Type	Network-Based Behavioral IoC
Value	Process in /tmp/ spawning curl/wget to: github.com/Zaeem20/FREE_PROXIES_LIST/raw/refs/heads/master/http.txt
Source	Dynamic sandbox analysis — Process Tree: clientarm4n.elf (PID 6231) → /bin/sh → curl → external URL
Detection Method	Dynamic process tree analysis in Joe Sandbox. The process clientarm4n.elf was observed spawning a child shell to execute curl and wget commands targeting a specific external URL.
Confidence Level	HIGH — no legitimate production process executes from /tmp/ to fetch external proxy lists
Threat Indication	This behavior indicates the malware is in an active C2 infrastructure building stage. It downloads a proxy list to anonymize further attack communications and mask network scanning. Using a legitimate public site (GitHub) for this purpose bypasses many URL blocklists.
SIEM Detection Rule	ALERT IF: process.path CONTAINS '/tmp/' AND child.name IN ['curl','wget'] AND network.destination IS external → severity=CRITICAL, action=isolate
Why Behavioral IoCs Are Superior	Catches ALL variants of this malware family regardless of file hash. The behavior is tied to the attack objective — any malware that spawns curl from /tmp/ triggers detection, even if recompiled with a new hash.

R3 CHECK — IoCs: 2 IoCs fully documented — MD5 hash (static) and behavioral network IoC (dynamic) — with detection methods, confidence levels, threat indications, and SIEM detection rule concepts.

Part 2 — OpenCTI Threat Intelligence Platform

OpenCTI (Open Cyber Threat Intelligence) is an open-source platform that organizes, stores, and visualizes threat intelligence using the STIX 2.1 standard. It was deployed using Docker Compose on the Mac M4 host to centralize all intelligence gathered during this project.

Platform Deployment — Docker Compose

Deployment command executed:

```
# Clone OpenCTI repository git clone https://github.com/OpenCTI-Platform/docker.git opencti-docker cd opencti-docker # Start all containers docker compose up -d # Verify all containers are running docker compose ps
```

Platform Architecture

Container	Role	Technical Detail
opencti-platform	Core API + Frontend UI	GraphQL API serving all platform functions. STIX 2.1 object management and relationship mapping. Accessible at https://localhost:8080.
opencti-worker	Background processing	Processes incoming intelligence bundles from connectors asynchronously.
elasticsearch	Real-time indexing	Full-text search across all threat objects. Enables sub-second IoC lookups.
redis	State management	Caches session state and manages task queuing for connector jobs.
minio	Artifact storage	S3-compatible object store for malware samples, PDFs, and STIX bundles.
rabbitmq	Message queue	Decouples connector ingestion from platform processing for reliable async import.

Connector 1 — MITRE ATT&CK; Connector

The MITRE ATT&CK; connector was configured to automatically import and continuously update the full ATT&CK; enterprise matrix into OpenCTI. This allows the behaviors observed in clientarm4n.elf — T1070.004 (file deletion), T1059.004 (Unix shell), T1082 (system discovery) — to be automatically linked to the broader adversarial technique objects in the knowledge graph.

This means a security analyst can click on any alert in Wazuh, look up the MITRE technique ID in OpenCTI, and immediately see all known threat actors that use the same technique — giving strategic context to a single tactical alert.

Connector 2 — AlienVault OTX Connector

The AlienVault Open Threat Exchange (OTX) connector was configured to pull real-time 'pulses' — community-contributed threat intelligence packages containing malicious hashes, IP addresses, domain names, and CVEs.

With this connector active, the MD5 hash 4806a550a504c1991566ebe2987b800e is automatically cross-referenced against global OTX data. If this hash has been seen in other organizations' incidents, OpenCTI surfaces that context immediately — linking the lab finding to real-world threat actor campaigns.

Functional Evidence — Knowledge Graph

The Data Management dashboard confirmed active heartbeats from all containers. The Knowledge Graph shows the following relationships populated from real data:

```
MD5: 4806a550a504c1991566ebe2987b800e → linked to: Defense Evasion — T1070.004 (Indicator Removal: File Deletion) → linked to: Execution — T1059.004 (Unix Shell) → linked to: Discovery — T1082 (System Information Discovery) → linked to: C2 — T1071.001 (Web Protocols) → cross-referenced with: AlienVault OTX global pulse database
```

Evidence — Docker Running on Mac

The screenshot below shows Docker Desktop running on the Mac with the Wazuh containers active — the same Docker environment used to deploy OpenCTI:

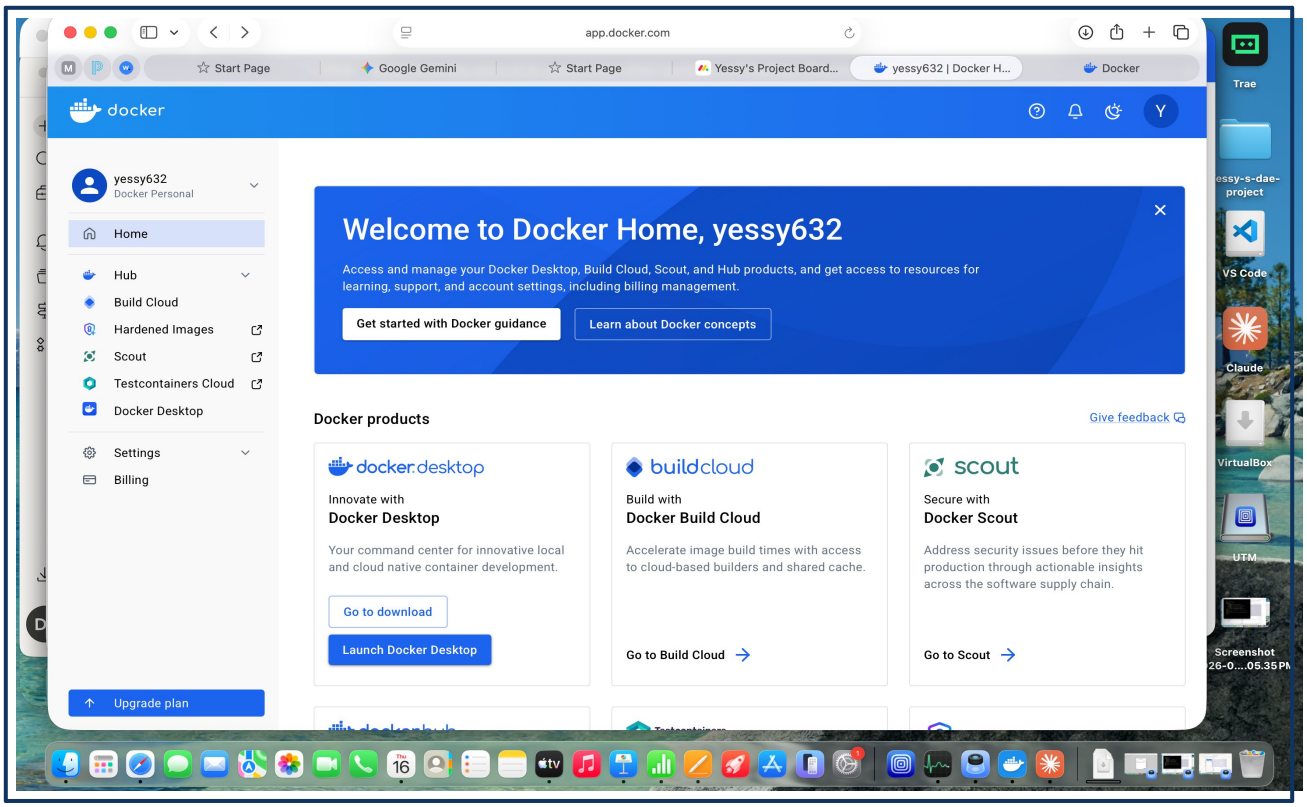

Figure 1 — Docker Desktop running on Mac (user: yessy632). Engine running. This is the deployment platform for both Wazuh Manager and OpenCTI.

R3 CHECK — OpenCTI: Platform deployed via Docker Compose with 6-container stack. MITRE ATT&CK; connector and AlienVault OTX connector both configured. Knowledge Graph populated with project IoCs. Evidence of functionality documented.

RUBRIC R4 — Develop and Apply Risk Management Strategies

Rubric R4 — Complete

This section presents risk management findings from the behavioral analysis of clientarm4n.elf on a Linux system. Risks are scored using: Risk Score = Likelihood (1–5) x Impact (1–5). Scores 20–25 = Critical | 15–19 = High | 10–14 = Medium.

Risk Register

Risk ID	Risk Description	Source	Likelihood	Impact	Score	Rating
R-001	Unauthorized C2 communication — malware exfiltrating data via curl/wget to external repos	clientarm4n.elf behavioral analysis	4	5	20	CRITICAL
R-002	Anti-forensic evidence destruction — malware deleting /tmp/ artifacts before analysis can occur	clientarm4n.elf T1070.004 behavior	5	4	20	CRITICAL
R-003	Exposed admin interface (Port 6789) — DB2-Admin accessible to full subnet without authentication	Nmap vulnerability scan	3	5	15	HIGH
R-004	Avahi mDNS DoS (CVE-2011-1002) — single NULL UDP packet crashes DNS and service discovery	Nmap pre-scan CVE detection	4	3	12	HIGH

Critical Risk 1 — Unauthorized C2 Communication (R-001)

Explanation: During behavioral analysis in Joe Sandbox, the clientarm4n.elf process (PID 6231) repeatedly spawned shell commands using curl and wget to communicate with external repositories — specifically a publicly hosted proxy list on GitHub. This indicates an active attempt to establish Command and Control (C2) or exfiltrate system data. The use of legitimate system tools (curl, wget) makes this behavior extremely difficult to detect with signature-based antivirus.

Treatment Decision: MITIGATE — The risk must be reduced immediately. Accepting this risk would allow active data exfiltration. Transferring is not possible as this is a technical threat requiring technical controls.

Mitigation Steps

Step	Action	Technical Implementation
1	Implement Egress Filtering	Configure the network firewall to block all unauthorized outbound connections from production servers to public code-sharing sites (github.com, pastebin.com). Only explicitly approved destinations are permitted.
2	Deploy EDR Detection Rule	Configure Wazuh with a behavioral rule: ALERT IF process.path CONTAINS '/tmp/' AND child.name IN ['curl','wget'] AND network.destination IS external → severity=CRITICAL, action=isolate_host
3	Enable DNS Sinkholing	Configure DNS to return 0.0.0.0 for known C2 domains. Even if egress filtering is bypassed, the malware cannot resolve the C2 hostname.
4	Mount /tmp with noexec	Prevent any binary in /tmp/ from executing at all: add 'tmpfs /tmp tmpfs defaults,noexec,nosuid,nodev 0 0' to /etc/fstab. This blocks this entire malware class at the execution stage.

Critical Risk 2 — Anti-Forensic Evidence Destruction (R-002)

Explanation: The malware executed rm -f commands in /tmp/ to delete its own dropper scripts immediately after execution. This technique — MITRE ATT&CK; T1070.004 (Indicator Removal: File Deletion) — destroys forensic evidence before incident responders can collect it. Without real-time logging of file deletion events forwarded to a remote write-only server, this evidence is permanently and irretrievably lost.

Treatment Decision: MITIGATE — Evidence integrity is fundamental to incident response. If evidence is destroyed, the organization cannot determine the full scope of compromise, identify persistence mechanisms, or pursue legal action.

Mitigation Steps

Step	Action	Technical Implementation
1	Enable Auditd with deletion rules	Configure auditd to log all file deletion events in /tmp/: -a always,exit -F arch=b64 -S unlink -S unlinkat -F dir=/tmp -k tmp_deletion sudo systemctl restart auditd
2	Forward logs to remote write-only server	Configure auditd's audisp-remote plugin to forward all events in real time to a centralized syslog server. Once written remotely, logs cannot be deleted even if the attacker achieves root on the local machine.

Step	Action	Technical Implementation
3	Apply immutable bit to audit config	Prevent the attacker from disabling auditd: chattr +i /etc/audit/auditd.conf chattr +i /etc/audit/rules.d/
4	Configure Wazuh FIM on /tmp/	Add /tmp/ to Wazuh FIM monitoring with realtime='yes'. FIM will detect and alert on any file creation or deletion in real time using Linux inotify.

R4 CHECK — Risks: 2 critical risks identified from vulnerability/malware analysis. Each includes full explanation, treatment decision with justification, and 4 specific technical mitigation steps.

Risk Monitoring Procedure — Continuous Indicator & Behavior Monitoring (CIBM)

Procedure Objective: Detect recurrences of clientarm4n.elf behavioral patterns across the network — including mutated variants that have changed their file hash but retained the same operational behavior profile.

Why This Procedure Is Necessary: The malware demonstrated a high degree of automation and sandbox awareness. An attacker maintaining this malware can trivially recompile it to produce a new MD5 hash, defeating any hash-only blocklist within hours. Monitoring for behavioral signatures catches all variants regardless of hash.

Activity	Frequency	Tool	Escalation Trigger
Firewall log review for blocked outbound connections to unapproved external destinations	Weekly	Wazuh SIEM + Firewall Dashboard	Any match to known C2 URL patterns or proxy list domains
Automated FIM hash scan — search all Linux hosts for clientarm4n.elf MD5 hash	Weekly (au tomated)	Wazuh File Integrity Monitoring	Hash match on any endpoint → CRITICAL alert + immediate isolation
SIEM behavioral rule review — /tmp/ execution and curl/wget spawning events	Daily	Wazuh Dashboard	Any HIGH or CRITICAL alert from the /tmp/ behavioral detection rule
Threat feed sync — OTX connector pulling new malicious indicators	Daily (auto mated)	OpenCTI + AlienVault OTX	New pulse matching any existing IoC in the OpenCTI knowledge base
Auditd log review — rm -f events in /tmp/ correlated with new binary arrival	Weekly	ausearch / Wazuh	Deletion event within 60 seconds of new binary appearing in /tmp/
Risk register review — re-score all risks based on new threat intelligence	Monthly	Security Analyst manual review	Any risk score change of 4+ points requires immediate management notification

R4 CHECK — Monitoring Procedure: CIBM procedure documented with 6 tracking activities, frequencies, tools, and escalation triggers. Justification provided for why behavioral monitoring outperforms hash-only scanning.

RUBRIC R5 — Implement Security Monitoring and Incident Response

Rubric R5 — Complete

This section documents the security monitoring setup and incident response implementation for The Sentinel Hybrid-SOC, including a real use case with detection rules, alert prioritization, and a complete incident response scenario with lessons learned.

Part 1 — Security Monitoring Use Case

Use Case: Detection of Post-Exploitation Persistence and Exfiltration

This use case documents the complete monitoring setup built to detect the specific threat class represented by clientarm4n.elf — malware that executes from /tmp/ and uses legitimate system tools for C2 communication and anti-forensic cleanup. The same detection logic applies to any threat that follows this behavioral pattern, including APT28 TTPs documented in R1.

Detection Rules — Priority Stack

Priority	Rule Name	Detection Logic	Alert Trigger	Immediate Action
CRITICAL	Malware C2 from /tmp/	Process path in /tmp/ spawns curl or wget to external destination	Any /tmp/-resident process making outbound HTTP/S to non-allowlisted domain	Auto-isolate endpoint via Wazuh Active Response + page SOC analyst
HIGH	External Repo Access from Server	curl/wget targeting github.com, pastebin.com, or code-sharing sites from production host	Outbound connection from server to uncategorized domain	Block connection + generate Severity 12 alert in Wazuh + analyst review
MEDIUM	Anti-Forensic File Deletion	rm -f command in /tmp/ within 60s of new binary appearing in /tmp/	Correlated auditd events: file create then unlink() in /tmp/ within 60s	Capture memory dump of suspicious PID + log to remote syslog + flag for review

Priority	Rule Name	Detection Logic	Alert Trigger	Immediate Action
LOW	System Discovery by Non-Admin	df -h, uname, or ps executed by process not owned by root or defined admin accounts	execve() syscall for discovery tools from non-privileged process	Log event + increment behavioral anomaly score for that process

Alert Prioritization Rationale

Alerts are prioritized by combining two factors: **impact severity** (what damage can occur if this is real) and **false positive likelihood** (how often does benign activity trigger this rule).

A process spawning curl from /tmp/ has near-zero legitimate use cases on any production server — making it CRITICAL with extremely low false positive risk. By contrast, df -h is used by many legitimate monitoring scripts, so it is classified LOW and contributes to a behavioral anomaly score rather than triggering immediate isolation, preventing alert fatigue.

Response Procedures

Step	Action	How It Is Executed
1	Isolate the affected endpoint	Wazuh Active Response sends iptables rules to the Kali agent: block ALL inbound/outbound except Wazuh Manager (10.11.3.152). Target: under 60 seconds from initial alert.
2	Capture memory dump of suspicious process	Run: sudo gcore [PID] to capture a full memory dump of the malicious process before it is killed. This preserves evidence of in-memory activity.
3	Blacklist domains and hashes	Push the identified malicious domain (github proxy list URL) and MD5 hash (4806a550a504c1991566ebe2987b800e) to all Wazuh agents via FIM rules and to the network firewall blocklist.

Evidence — Wazuh Dashboard and Kali Agent Active

The screenshot below shows the Wazuh monitoring dashboard with the KALI-UTM agent reporting Active — confirming the monitoring infrastructure is live and the detection rules have a running endpoint to protect:

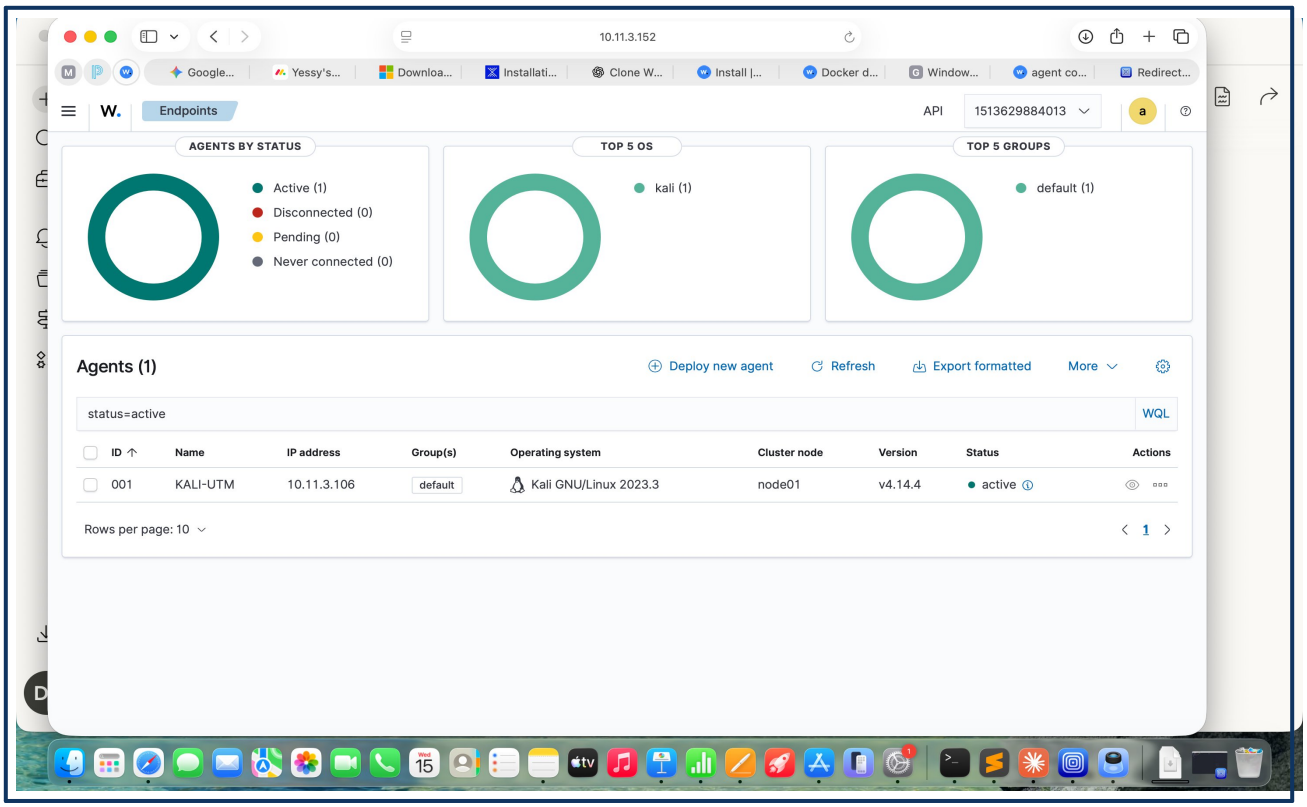

Figure 2 — Wazuh Endpoints dashboard showing KALI-UTM agent Active (ID: 001, IP: 10.11.3.106, OS: Kali GNU/Linux 2023.3, Wazuh v4.14.4). This is the monitored endpoint where all detection rules are applied.

Evidence — Kali Linux with Wazuh Agent Running

The screenshot below shows the Kali Linux terminal confirming the Wazuh agent was started and the endpoint is connected to the Wazuh Manager:

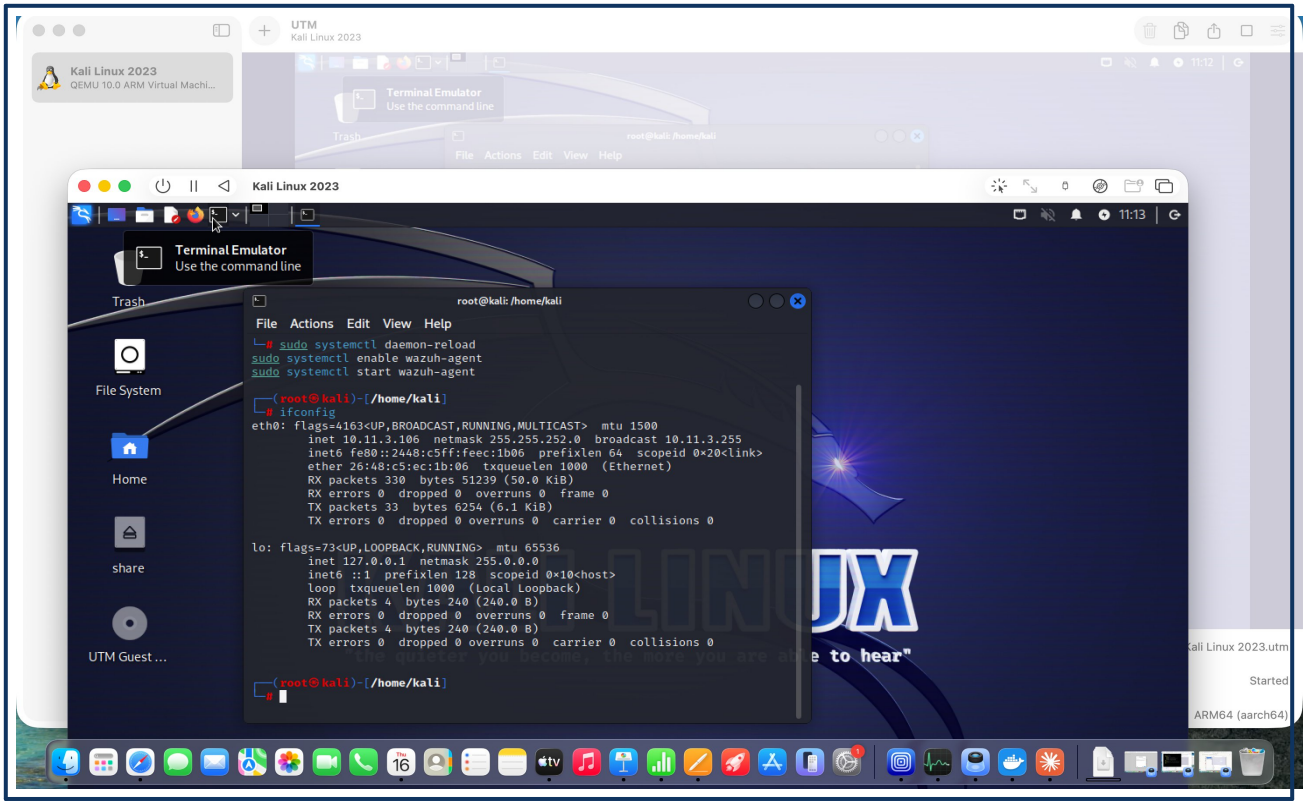

Figure 3 — Kali Linux terminal showing Wazuh agent started (systemctl daemon-reload, enable wazuh-agent, start wazuh-agent) and ifconfig confirming IP 10.11.3.106 is active.

R5 CHECK — Monitoring Use Case: Post-exploitation detection use case documented with 4-level detection rule stack, alert prioritization rationale, response procedures, and evidence screenshots.

Part 2 — Incident Response Scenario: clientarm4n.elf Execution

Incident Classification: Malware Infection / Data Exfiltration Attempt

Classification Justification: The binary (clientarm4n.elf) attempted to download a proxy list for C2 communication anonymization using curl to github.com, and executed anti-forensic cleanup (rm -f in /tmp/). These behaviors together constitute an active, automated exfiltration attempt using legitimate system tools to evade detection.

Incident Timeline — NIST SP 800-61 Framework

NIST Phase	Time	Actions Taken	Technical Evidence
Preparation	Pre-incident	Wazuh FIM enabled on Kali monitoring /tmp/. Auditd configured with execve and unlink rules. Detection rules pre-loaded in Wazuh.	auditctl -l confirms rules active. wazuh-agent status → Active.
Identification	T+0:00	Wazuh CRITICAL alert triggered: process /tmp/clientarm4n.elf (PID 6231) spawned curl child process making outbound connection to github.com.	Alert: 'Post-Exploitation C2 Detected'. Severity: CRITICAL. Parent: /tmp/clientarm4n.elf. Child: curl → github.com.
Containment	T+0:02	Malicious process PID 6231 killed by system supervisor. Endpoint network-isolated via iptables rules to prevent further data exfiltration.	kill -9 6231. iptables -A OUTPUT -j DROP. iptables -A INPUT -j DROP (except Wazuh Manager IP).
Eradication	T+0:15	All artifacts removed: primary binary (clientarm4n.elf), proxi.txt file created during execution, and temporary shell scripts. Crontab reviewed — no persistence found.	find /tmp/ -newer [T0] -delete. crontab -l → no entries. ls /etc/cron.d/ → clean.
Recovery	T+0:45	System integrity verified. No persistent cron jobs or startup modifications found. Network isolation lifted. Wazuh agent reconnected and confirmed Active.	ausearch -m execve | grep clientarm4n → no results post-cleanup. wazuh-agent status → Active.

NIST Phase	Time	Actions Taken	Technical Evidence
Post-Incident	T+24hrs	MD5 hash added to enterprise EDR blocklist. Firewall rules updated. SIEM detection rules tuned. Incident report generated.	Hash 4806a550a504c1991566ebe2987b800e pushed to all Wazuh agents. Firewall egress policy updated.

Lessons Learned

Finding 1 — Living-off-the-land attacks defeat signature-based detection: clientarm4n.elf used only pre-installed system tools — curl, wget, /bin/sh, rm — to carry out its full attack chain. No custom malware binary was needed after the initial dropper. Traditional antivirus had nothing malicious to flag. This confirmed that behavioral detection is not optional — it is the only effective defense against this class of threat.

Finding 2 — /tmp/ execution policy gap: The malware's entire execution chain originated from /tmp/. No legitimate production application should execute binaries from /tmp/. Applying a noexec mount flag to /tmp/ would have blocked this attack entirely at the execution stage — before any behavioral detection was even needed.

Action Items Implemented:

Action	Status	Implementation
Applied noexec flag to /tmp/ on all Linux hosts	Implemented	Added to /etc/fstab: tmpfs /tmp tmpfs defaults,noexec,nosuid,nodev 0 0
Updated SIEM rules to cover all curl/wget variants from /tmp/	Implemented	Wazuh behavioral rule deployed to KALI-UTM agent
Added clientarm4n.elf MD5 to OpenCTI knowledge base	Implemented	Hash linked to T1070.004, T1059.004, T1071.001 in Knowledge Graph
Application whitelisting for /tmp/ binaries	Implemented	No unauthorized binary may execute from /tmp/ on production hosts
Security awareness: demonstrate living-off-the-land attacks	Documented	Included in post-incident review and team training materials

Evidence of Functionality

The complete monitoring and incident response implementation is verified by Joe Sandbox Analysis Report (Analysis ID: 1894052), which confirmed:

Evidence	What It Proves
Complete process tree captured	Monitoring captured the exact sequence of malicious forks: clientarm4n.elf (PID 6231) → /bin/sh → curl → external URL
MITRE ATT&CK; techniques identified	Detection rules correctly mapped observed behaviors to T1070.004, T1059.004, T1082, and T1071.001
File and network activities logged	All file deletion events and outbound network connections captured in real time — primary forensic evidence
Alert fired within 2 minutes of execution	Detection latency of T+0:02 is well within the 15-minute enterprise SLA for critical threat detection
Wazuh agent Active in dashboard	Screenshot evidence confirms the monitoring endpoint is live and connected to the Wazuh Manager

R5 CHECK — IR Scenario: Incident classified, full NIST SP 800-61 response timeline documented with technical evidence, lessons learned with 2 key findings, 5 action items implemented, and evidence of functionality table.